PROJECT PLAN

Campus Gate Security System

Barcode & Biometric Student Verification at Entry Points

Software Engineering Course | Version 2.0 | April 2026
Remaining Duration: 8 Weeks (40 Working Days)

PROJECT TEAM ROLES
Team Leader | **Architect** | **Designer** | **Programmer**

1. Introduction

1.1 Background and History

Campus security has become an increasingly critical concern at educational institutions. The institution has experienced a notable rise in incidents including theft of student property and cases of sexual assault linked to unauthorized individuals gaining access to campus. A key vulnerability identified by security officers is the lack of a reliable, automated mechanism for distinguishing enrolled students from non-students at entry points. Manual verification by guards is subject to human error, inconsistency during peak hours, and susceptibility to social engineering.

The institution already issues each student a unique ID card embedded with a barcode, yet this card is not integrated with any automated access control system. This project leverages that existing infrastructure to build an end-to-end gate security solution.

1.2 Aims and Objectives

The primary aim of this project is to design and develop a Campus Gate Security System (CGSS) that automatically verifies the identity of all individuals seeking campus entry. The specific objectives are:

- To integrate existing student ID barcodes with a real-time database verification system at campus gates.
- To implement fingerprint biometric verification as a secondary authentication layer for students.
- To design a visitor management module enabling pre-registration, national ID logging, and time-limited QR access credentials.
- To provide a real-time security dashboard and automated alert system for security personnel.
- To maintain a full, tamper-evident audit log of all access events to support post-incident investigations.

1.3 Team Roles and Responsibilities

This project is executed by a team of four members. Each member owns a clearly defined functional domain across all project phases:

Role	Name	Primary Responsibilities	Tools / Deliverables
Team Leader	TBD	Project planning and scheduling; stakeholder communication; risk management; milestone tracking; chairing meetings; final sign-off on all deliverables; UAT coordination.	Project Plan, Meeting Minutes, Risk Register, UAT Sign-off
Architect	TBD	System architecture design; technology stack selection; database schema design; API contract definition; ensuring non-functional requirements (security, scalability) are met; code review.	Architecture Diagram, ERD, API Contract, DB Schema
Designer	TBD	Requirements elicitation through user interviews; use case modeling; UI/UX wireframing; gate controller interface design; visitor portal and dashboard visual design; user training materials.	SRS, Use Case Diagrams, Wireframes (Figma), Style Guide
Programmer	TBD	Backend API development; database implementation; barcode and fingerprint SDK integration; visitor management module; alert system; frontend gate controller and dashboard coding; unit and integration testing.	Source Code, Unit Tests, Integrated System

1.4 Project Summary

The CGSS is a full-stack software solution consisting of: (1) a gate-facing controller application interfacing with barcode scanners and fingerprint readers; (2) a backend REST API connected to the institutional student database; (3) a visitor pre-registration portal; and (4) a real-time security operations dashboard. With 8 weeks remaining, the project follows an incremental delivery model across three functional increments, culminating in a UAT-validated go-live at the main campus gate.

2. User / Client Involvement

2.1 Client and User Stakeholders

The primary client is the Campus Security Department, acting on behalf of the institution's administration. The Designer is responsible for maintaining the client relationship, conducting interviews, and presenting mockups for sign-off. Key user groups include: gate security guards (daily operators), academic and administrative staff, enrolled students, and the IT Department (hosting and maintenance).

2.2 Resources Required from Client

Resource	Details	Project Team Contact	Required By
Student database read access	Read-only API or export of active student records (ID, name, barcode, enrolment status)	Architect	Week 1
Fingerprint enrollment data	Existing biometric records if any; otherwise enrollment scope to be agreed	Architect + Programmer	Week 2
Physical hardware provision	Barcode scanners, fingerprint readers, gate-mounted terminals	Programmer	Week 3
Server / hosting environment	On-premise or cloud server for staging and production deployment	Architect	Week 7
Security staff for UAT	At least 3 guards available for hands-on testing at gate terminal	Team Leader	Week 8

Policy and legal documents	Existing access policies, visitor rules, POPIA / data privacy regulations	Designer	Week 1
Designated client liaison	Single point of contact with authority to approve deliverables	Team Leader	Immediately

2.3 Client Review Points

The Team Leader will schedule formal client review sessions at the end of Weeks 2, 5, and 7. The Designer will prepare mockups and demo builds for each review. Written sign-off from the client liaison is required before the project advances past each milestone.

3. Risks

The Team Leader is responsible for maintaining the risk register and escalating risks that exceed a threshold of High Likelihood + High Impact. All risks are reviewed at weekly team meetings.

#	Risk Description	Likelihood	Impact	Owner	Mitigation Strategy
R1	Delayed access to student database	Medium	High	Architect	Negotiate access in Week 1; use anonymised dummy data during development.
R2	Fingerprint hardware incompatibility with SDK	Medium	High	Programmer	Prototype hardware integration in Week 2; select SDKs with broad device support (DigitalPersona / Mantra).
R3	Student/staff resistance to biometric data collection	High	Medium	Team Leader + Designer	Consult legal/compliance; make biometrics optional with ID barcode-only fallback mode.
R4	Breach of biometric or personal data	Low	Very High	Architect	Encrypt all biometric templates (AES-256); enforce HTTPS/TLS; comply with POPIA.

R5	Gate hardware failure during operation	Medium	High	Programmer	Implement offline caching mode with manual guard override and exception logging.
R6	Scope creep from additional stakeholder requests	High	Medium	Team Leader	Freeze requirements after Week 1; enforce formal change request process (Section 12).
R7	Poor network connectivity at gate location	Medium	Medium	Architect	Design local-cache sync architecture so terminal operates offline and syncs when connected.
R8	8-week timeline overrun	Medium	High	Team Leader	Weekly milestone tracking; re-plan tasks immediately on any slippage detected.
R9	Guards resist adopting the new system	Medium	Medium	Designer	Involve guards in UAT from Week 8; run a dedicated 2-hour training session.
R10	Team member unavailability	Low	High	Team Leader	Cross-train all members; maintain up-to-date documentation so any member can cover another.

4. Standards, Guidelines, and Procedures

4.1 Coding Standards (Programmer + Architect)

- All code follows community conventions: PEP 8 for Python; Airbnb style guide for JavaScript/React.
- All functions include docstrings stating purpose, parameters, and return values.
- No hardcoded credentials or secrets in source code. All sensitive config stored in environment variables.
- Minimum 70% unit test coverage on all backend modules before integration (enforced by CI pipeline).

4.2 Documentation Standards (Designer + Team Leader)

- All project documents follow the IEEE 830 SRS template structure.
- Meeting minutes are recorded by the Team Leader and distributed within 24 hours.

- All diagrams (use cases, ERDs, architecture) are produced in Draw.io and exported to PDF for submission.
- The project wiki (GitHub Wiki) is updated at the end of every week by the responsible role.

4.3 Version Control (Programmer + Architect)

- Git is used for all source code, hosted on GitHub (private repository).
- Branching strategy: main (production-ready), develop (integration), feature/[task-id] (per developer per task).
- No direct commits to main. All merges require a pull request reviewed by the Architect before merging.
- Commit message format: [type]: description (e.g., feat: add barcode scanner API endpoint).

4.4 Security and Privacy Standards (Architect)

- System must comply with the Protection of Personal Information Act (POPIA) for biometric data.
- Biometric templates stored as encrypted mathematical representations only — never as raw images.
- All client-server communication must use HTTPS/TLS 1.2 or higher.
- Admin dashboard access requires role-based JWT authentication with strong password enforcement.

4.5 Meeting and Communication Procedures (Team Leader)

- Weekly stand-up: every Monday, 15 minutes, chaired by the Team Leader. Agenda: progress, blockers, next steps.
- Formal milestone review meetings with the client at the end of Weeks 2, 5, and 7.
- All team communication via a designated group channel; formal client correspondence via email from the Team Leader.

5. Organization of the Project

5.1 Relation to Other Projects

The CGSS interfaces with, but does not modify, the institution's existing Student Information System (SIS). The Architect will negotiate a read-only API contract with the IT Department to consume student data. Network infrastructure is a dependency: the Architect and IT Department must confirm gate-terminal connectivity before hardware installation in Week 3.

5.2 Role Responsibilities Per Project Phase

Project Phase	Team Leader	Architect	Designer	Programmer
Requirements & Analysis	Approve scope, schedule stakeholder meetings	Define system boundaries and technical constraints	Conduct user interviews, produce use cases and DFDs	Provide input on technical feasibility
System Design	Review and approve all design documents	Design system architecture, DB schema, API contracts	Produce UI wireframes, user journey maps	Review DB schema for implementation feasibility
Increment 1: Student Verification	Track progress, resolve blockers	Guide SDK integration, review code structure	Develop gate controller UI screens	Build barcode and fingerprint modules, REST API
Increment 2: Visitor Management	Review increment demo with client	Review visitor data model and QR security	Design visitor portal and badge print layout	Build visitor registration module, QR generation
Increment 3: Dashboard & Alerts	Review increment demo with client	Review real-time architecture (WebSocket)	Design dashboard layout and alert UI	Build dashboard, alert engine, blacklist module
Integration & System Testing	Escalate critical defects, approve test report	Review test architecture, validate security testing	Prepare UAT test scenarios for guards	Execute integration, system, and regression tests
UAT & Deployment	Chair UAT sessions, obtain client sign-off	Supervise production deployment and server config	Conduct guard training, prepare user manual	Fix UAT defects, support deployment

5.3 Training Schedule

Who	Training Required	Delivered By	Scheduled Week
Programmer	Fingerprint SDK API (DigitalPersona / Mantra MFS100)	Self-study + Architect guidance	Week 2
Programmer	React WebSocket integration (Socket.IO)	Self-study	Week 2

Designer	Figma advanced prototyping and user testing	Self-study	Week 1
Architect	Cloud server deployment and Docker	Self-study	Week 7
Security Guards (users)	Operating the gate controller app and override procedures	Designer (training materials) + Team Leader	Week 8 (pre-UAT)

6. Project Phases

6.1 Life Cycle Model

This project adopts the Incremental Development Model. The system is delivered in three functional increments (Student Verification, Visitor Management, Security Dashboard), allowing client review and feedback after each increment before the next begins. This model is appropriate given the modular nature of the three subsystems, the 8-week deadline, and the need for early client validation before full commitment to all features.

6.2 Task Dependency Table

The table below defines each task, its duration, dependencies, cost, and the responsible team member(s). This forms the basis for both the Gantt chart and the PERT diagram that follow, as referenced from Cooper [1984] and the course lecture notes on project planning.

Task	Description	Duration (days)	Predecessor(s)	Cost/Day (E)	Total Cost (E)	Responsible
A	Requirements Analysis	5	—	200	1,000	Team Leader + Architect
B	Architecture Design	5	A	200	1,000	Architect
C	UI/UX Wireframes	10	A	150	1,500	Designer
D	Database & Backend Setup	5	A	180	900	Programmer + Architect
E	Barcode & Fingerprint Module	10	B, D	180	1,800	Programmer

F	Gate Controller UI Development	8	C, D	150	1,200	Designer + Programmer
G	Visitor Management Module	5	E	180	900	Programmer
H	Visitor Portal UI	5	F, G	150	750	Designer
I	Security Dashboard	5	F, G	170	850	Designer + Programmer
J	Alert & Blacklist Module	5	E, G	180	900	Programmer
K	Integration & System Testing	5	H, I, J	160	800	Team Leader + Programmer
L	UAT & Deployment	5	K	200	1,000	Team Leader (All)
	TOTALS	68 days			11,600	

6.3 Gantt Chart — 8-Week Schedule

The Gantt chart below shows the planned schedule across 8 weeks (40 working days). Bar colours indicate: Critical Path tasks (red), Concurrent tasks with slack (green), and standard scheduled tasks (blue). EST = Earliest Start Time (days); LST = Latest Start Time (days); Slack = LST − EST.

ID	Task	Who	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	EST	LST	Slack
A	Requirements Analysis	Team Leader, Architect									0	0	0
B	Architecture Design	Architect									5	5	0
C	UI/UX Wireframes	Designer									5	13	8
D	Database & Backend Setup	Programmer, Architect									5	5	0
E	Barcode & Fingerprint Module	Programmer									10	10	0
F	Gate Controller UI	Designer, Programmer									10	18	8

G	Visitor Management Module	Programmer									20	20	0
H	Visitor Portal UI	Designer									25	25	0
I	Security Dashboard	Designer, Programmer									25	25	0
J	Alert & Blacklist Module	Programmer									25	25	0
K	Integration & System Tests	Team Leader, Programmer									30	30	0
L	UAT & Deployment	Team Leader (All roles)									35	35	0

■ **Red bars** = Critical Path (slack = 0) ■ **Green bars** = Concurrent tasks (slack ≥ 8 days) ■ **Blue bars** = Standard scheduled tasks

6.4 PERT Analysis

PERT (Program Evaluation and Review Technique) identifies the Earliest Start Time (EST), Latest Start Time (LST), and Slack for each task. Slack = LST − EST. Tasks with a Slack of 0 form the Critical Path: any delay on these tasks delays the entire project.

Task	Description	Duration (days)	Predecessor(s)	EST	ECT	LST	LCT	Slack
A	Requirements Analysis	5	—	0	5	0	5	0
B	Architecture Design	5	A	5	10	5	10	0
C	UI/UX Wireframes	10	A	5	15	13	18	8
D	Database & Backend Setup	5	A	5	10	5	10	0
E	Barcode & Fingerprint Module	10	B, D	10	20	10	20	0
F	Gate Controller UI	8	C, D	10	18	18	26	8
G	Visitor Management Module	5	E	20	25	20	25	0
H	Visitor Portal UI	5	F, G	25	30	25	30	0

I	Security Dashboard	5	F, G	25	30	25	30	0
J	Alert & Blacklist Module	5	E, G	25	30	25	30	0
K	Integration & System Tests	5	H, I, J	30	35	30	35	0
L	UAT & Deployment	5	K	35	40	35	40	0

Critical Path: A → B → E → G → H → K → L OR A → D → E → G → I → K → L

Earliest Completion Time (ECT): 40 working days (8 weeks). *Tasks C and F have 8 days of slack and may be delayed without impacting the overall deadline.*

6.5 Milestones

Milestone	End of Week	How Verified	Owner
M1: Requirements Approved	Week 1	Signed SRS document approved by client liaison	Team Leader
M2: Design Complete	Week 2	Architecture diagram, DB schema, wireframes reviewed and approved by client	Architect + Designer
M3: Increment 1 Complete (Student Verification)	Week 4	Barcode scan + fingerprint verification functional at gate terminal; demo to client	Programmer
M4: Increment 2 Complete (Visitor Management)	Week 5	Visitor pre-registration and QR code entry validated end-to-end	Programmer + Designer
M5: Increment 3 Complete (Dashboard & Alerts)	Week 6	Live dashboard showing real-time entry events; alerts fire on blacklisted scan	Programmer + Designer
M6: System Test Passed	Week 7	All test cases pass; zero Critical defects open; test report signed by Team Leader	Programmer + Team Leader
M7: Go-Live	Week 8	Client sign-off on UAT; system live at main campus gate	Team Leader (All)

7. Requirements Analysis and Design

7.1 Methods and Techniques

The Designer leads all requirements elicitation and modeling activities in Week 1, supported by the Architect for technical feasibility assessments:

- Structured interviews with campus security guards, students, and the IT Department (conducted by the Designer).
- Use Case Diagrams (UML) modeling system behavior from each user role's perspective (Designer).
- Data Flow Diagrams (DFD Level 0 and Level 1) to show data movement between gate scanners, the API, and the database (Designer + Architect).
- Entity-Relationship Diagrams (ERD) to model the data schema: Students, Visitors, Entry Logs, Alerts, Blacklist (Architect).
- Non-functional requirements specification: performance targets, security constraints, uptime SLA (Architect).
- UI wireframing in Figma for the gate controller app, visitor portal, and dashboard (Designer).
- Requirements Traceability Matrix: maps each requirement to a design element and a test case (Designer + Team Leader).

7.2 Tools

Tool	Purpose	Used By
Draw.io	Use Case Diagrams, ERDs, DFDs, PERT diagrams, Architecture diagrams	Designer + Architect
Figma	UI wireframes and interactive prototype for client walkthroughs	Designer
Google Docs / Word	Software Requirements Specification (SRS) document authoring	Designer + Team Leader
GitHub Wiki	Centralised, version-controlled project documentation	All (Team Leader oversees)
Miro / Whiteboard	Collaborative brainstorming sessions and initial task breakdown	All

8. Implementation

8.1 Technology Stack

Layer	Technology	Justification	Implemented By
Backend API	Node.js + Express.js	Lightweight, fast, large npm ecosystem for REST APIs	Programmer
Database	PostgreSQL	Robust relational DB for structured student/log data	Programmer + Architect
Frontend (Gate UI & Dashboard)	React.js	Component-based, supports real-time updates via WebSocket	Designer + Programmer
Fingerprint SDK	DigitalPersona U.are.U SDK or Mantra MFS100	Broad device support, good documentation, cost-effective	Programmer
Barcode Scanning	ZXing library (webcam) or USB HID scanner input	Compatible with existing student ID barcode formats	Programmer
QR Code Generation	qrcode (npm library)	Visitor one-time QR codes for pre-registered entries	Programmer
Real-time Updates	Socket.IO	Live dashboard updates for security personnel	Programmer
Authentication	JWT (JSON Web Tokens)	Stateless, secure session management for admin users	Programmer + Architect
Encryption	AES-256 (biometric templates), bcrypt (passwords)	Industry-standard encryption for sensitive data	Architect + Programmer
Containerisation	Docker	Consistent dev/staging/production environments	Architect
CI Pipeline	GitHub Actions	Auto-run unit tests on every pull request; block merges below 70% coverage	Architect

8.2 Implementation Approach

- **Code Ownership:** The Programmer develops all backend and frontend code. The Architect reviews all pull requests before merging to develop.

- **Design Handoff:** The Designer delivers all Figma wireframes as specification before each increment begins so the Programmer has clear UI targets.
- **Progress Tracking:** The Team Leader monitors the GitHub project board weekly and flags any tasks that are behind schedule.
- **Environment Consistency:** Docker containers ensure the Programmer's local environment matches the staging server; the Architect manages the Dockerfile.

9. Testing

9.1 Test Environment

- A dedicated staging server (provisioned by Architect in Week 7) replicating the production environment with the same OS, database schema, and hardware connections.
- A test dataset of at least 50 anonymised student records loaded by the Architect into the staging database.
- A physical test bench set up by the Programmer (Week 3 onward) with gate terminal, USB barcode scanner, and fingerprint reader.

9.2 Types of Testing and Ownership

Test Type	Description	Owner	When
Unit Testing	Individual functions tested in isolation (e.g., barcode decoder, DB lookup functions)	Programmer	During each increment (Weeks 3–6)
Integration Testing	Scanner → API → Database full flow verified end-to-end	Programmer + Architect	After each increment
System Testing	Full system tested against all SRS requirements	Programmer + Team Leader	Week 7
Performance Testing	Verify system processes ≥30 verifications/minute without latency	Programmer	Week 7
Security Testing	Attempt SQL injection, unauthorised API access, data interception tests	Architect	Week 7

User Acceptance Testing (UAT)	Security guards test the system in realistic gate scenarios	Team Leader (facilitated); Designer (training)	Week 8
Regression Testing	Re-run all tests after any bug fix or change is applied	Programmer	Ongoing (Weeks 7–8)

9.3 Integration and Testing Order

The Programmer executes integration in the following sequence, with the Architect reviewing each stage:

- Step 1 (Week 3): Unit test barcode scanner module and student DB lookup in isolation.
- Step 2 (Week 4): Integrate barcode + fingerprint modules with the API; test combined student verification end-to-end.
- Step 3 (Week 5): Integrate visitor management module; test QR code generation and gate validation.
- Step 4 (Week 6): Integrate dashboard, alert engine, and blacklist module; test real-time updates.
- Step 5 (Week 7): Full system test across all three increments simultaneously; performance and security tests.
- Step 6 (Week 8): UAT with live gate hardware; guards execute real-world entry scenarios.

9.4 Testing Procedures

- Each test case documents: Test ID, objective, preconditions, steps, expected result, actual result, and pass/fail status.
- Defects logged immediately in GitHub Issues by the Programmer with severity: Critical / Major / Minor.
- No increment advances to the next phase if any Critical defects are open (enforced by Team Leader).
- The Team Leader compiles a formal Test Report after each testing cycle and presents it to the client.

10. Resources

10.1 Hardware Resources

Resource	Purpose	Qty	Responsible for Procurement
Gate terminal (PC or tablet)	Runs the gate controller application at the entry point	1 (pilot gate)	Client (IT Department)

USB Barcode Scanner	Scans student ID barcodes at the gate	1 per terminal	Client
USB Fingerprint Reader (Mantra MFS100 or equivalent)	Captures and matches student fingerprints	1 per terminal	Client
Development laptops	Each team member's personal development machine	4	Each team member
Staging server	Hosts test environment for integration and system testing	1	Architect (provisioned Week 7)
Production server	Hosts the live system post-deployment	1	Client (IT Department)

10.2 Software Resources

Software	Cost	Used By
Node.js, React.js, PostgreSQL	Free (open-source)	Programmer
GitHub (private repo + Actions CI)	Free (academic)	All (Architect manages)
Docker Desktop	Free	Architect + Programmer
Figma	Free tier	Designer
Draw.io	Free (web-based)	Designer + Architect
Fingerprint SDK (DigitalPersona / Mantra)	Evaluation licence for dev; production licence by client	Programmer
Visual Studio Code / JetBrains IDE	Free / student licence	Programmer + Architect

11. Quality Assurance

11.1 Quality Goals

Quality Attribute	Target	How Measured	Verified By
Reliability	99.5% uptime during gate operational hours (06:00–22:00)	Uptime monitoring on staging server	Architect
Performance	Identity verification completed in ≤3 seconds per person	Load test with simulated 30 scans/minute	Programmer
Security	No unauthorised access to student or biometric data	Penetration testing and code audit	Architect
Usability	Guard operates gate terminal unaided after 2-hour training	UAT observation — guard completes 10 tasks without help	Designer + Team Leader
Maintainability	Any developer can understand and modify any module within 30 minutes	Code review assessment by Architect	Architect
Accuracy	Barcode + fingerprint verification rejects non-students with 0% false acceptance rate	Test with 50+ real and fake ID combinations	Programmer

11.2 QA Procedures

- **Code Review:** Code Review (Architect): Every pull request reviewed before merging. Checklist: correctness, standards compliance, security vulnerabilities, test coverage ≥70%.
- **Automated CI:** CI Pipeline (Architect): GitHub Actions runs unit tests automatically on every PR. Builds failing tests or coverage thresholds are blocked from merging.
- **Weekly Status:** Weekly QA Status (Team Leader): A brief defect status update is presented at each Monday stand-up by the Programmer.
- **Phase Gates:** Phase Gate Reviews (Team Leader): Formal review at the end of each phase confirms deliverables meet acceptance criteria before advancing.
- **UAT Sign-Off:** UAT Sign-Off (Team Leader): The client representative must formally sign the UAT results document before system go-live is authorised.

11.3 Quality Documents

The following quality documents will be maintained: (1) Software Requirements Specification — Designer; (2) Test Plan and Test Cases — Programmer; (3) Test Results Report — Team Leader; (4) Defect Log — Programmer (GitHub Issues); (5) Code Review Checklists — Architect; (6) UAT Sign-off Document — Team Leader.

12. Changes

12.1 Change Management Philosophy

Given the tight 8-week schedule, uncontrolled changes are a significant threat to delivery. This project enforces a formal Change Control Process to ensure all changes are evaluated, approved, and tracked before implementation. The Team Leader is the final authority on all change approvals.

12.2 Change Request Procedure

Step	Action	Responsible
1. Submit	Requestor completes a Change Request Form (CRF) stating: what changes, why, and proposed new approach. Submitted to Team Leader via email.	Requestor (any team member or client)
2. Assess Impact	The Architect assesses technical impact; the Designer assesses design impact. Written impact assessment produced within 2 working days.	Architect + Designer
3. Classify	Team Leader classifies change: Minor (no schedule/budget impact) or Major (requires re-planning or extra effort).	Team Leader
4. Approve	Minor changes: Team Leader approves alone. Major changes: require sign-off from both Team Leader and client liaison.	Team Leader + Client (if Major)
5. Implement	Approved changes added to GitHub Issues, assigned to the Programmer (or Designer), and implemented on a new feature branch.	Programmer / Designer
6. Test	Programmer runs regression tests to confirm the change meets its objective and has not broken existing functionality.	Programmer

7. Document	The Architect logs the approved change in the Configuration Management Log with version, date, description, and approver.	Architect

12.3 Configuration Management

- All source code and documentation version-controlled in GitHub (managed by Architect).
- Document versioning convention: v[major].[minor] — e.g., v1.0 initial, v1.1 minor update, v2.0 major revision.
- Baseline versions of the SRS, Design Document, and Test Plan are tagged in Git by the Architect and protected against direct modification.
- Configuration Management Log maintained by the Architect, listing every approved change with version, description, impact, approver, and date.

End of Project Plan | Version 2.0 | April 2026 | 8-Week Schedule